

1. Importação de Dados (Regra R8)

O que fizemos: O professor pediu para não escrevermos o nome das turmas ou professores diretamente no código (nada de dados "hardcoded"). O código tem de funcionar para qualquer escola. Por isso, criámos uma função genérica usando a biblioteca `pandas`. Ela vai à pasta, lê os ficheiros CSV e guarda tudo em tabelas virtuais chamadas `DataFrames`. Se o ficheiro CSV mudar, o modelo adapta-se logo.

2. Modelação do Problema (Regras R1 a R7)

Variáveis e Exceções (R6)

A nossa ideia: O solver precisa de variáveis booleanas (0 ou 1) que cruzam: Turma + Disciplina + Dia + Período + Sala. Se for 1, a aula acontece ali. Se for 0, não acontece. Para a regra **R6** (professores indisponíveis), em vez de criarmos milhares de variáveis que dão zero, otimizámos o processo: o código vê a lista de exceções e **nem sequer cria** as variáveis nessas horas. O solver fica muito mais rápido assim.

Regra R1: Limite de turmas

A nossa ideia: Uma turma não se pode dividir ao meio. Fixamos uma turma, um dia e uma hora. A soma de todas as aulas possíveis nesse momento não pode passar de 1 (no máximo têm uma aula).

Regra R2: Carga Horária Semanal

A nossa ideia: A tabela diz que a turma tem X horas de Matemática por semana. Fomos buscar todas as variáveis de Matemática dessa turma na semana toda e dissemos ao solver: a soma disto tudo tem de ser estritamente igual àquele número X.

Regra R3: Espalhar as aulas (Limite diário)

A nossa ideia: Ninguém quer ter 4 horas de Português no mesmo dia. Então, se a disciplina for "normal", o limite diário é 1 aula. Se a tabela disser que tem `duplo_periodo`, o limite passa a 2 aulas por dia. Isto espalha as aulas pela semana.

Regra R4: Blocos Duplos

A nossa ideia: Algumas disciplinas práticas precisam de 2 tempos seguidos (ex: Físico-Química). Um tempo solto não serve. Dissemos ao modelo: "Se meteres uma aula no período P, és obrigado a meter outra colar a ela, ou no P-1 ou no P+1".

Regra R5: Professores

A nossa ideia: Um professor só pode estar numa sala. Num determinado dia e período, somamos todas as aulas que aquele professor pode dar a todas as turmas. A soma final não pode passar de 1.

Regra R7: As Salas

A nossa ideia: Para a tipologia (só Ginásio para Educação Física), resolvemos isso filtrando logo no início. Para a lotação física, somamos todas as turmas que tentam entrar na mesma sala à mesma hora, e limitamos a 1.

Status da resolução inicial: Ótimo/Viável

Demonstração - Horário da 7ªA:

↶ Search...

☐	Index0 int64	Seg object	Ter object	Qua object	Qui object
☐	1	Matemática (Sala Normal_1)	Matemática (Sala Normal_1)	Inglês (Sala Normal_2)	Ciências (Laborató
☐	2	-	-	-	-
☐	3	-	-	História (Sala Normal_6)	-
☐	4	Português (Sala Normal_3)	Português (Sala Normal_2)	Português (Sala Normal_3)	Matemática (Sala N
☐	5	Ciências (Laboratório_1)	Inglês (Sala Normal_1)	Matemática (Sala Normal_4)	Português (Sala No

5 rows, 5 columns
No selection

3. Otimização (O1) - Minimizar Buracos dos Professores

A nossa ideia: Para evitar que os professores fiquem a apanhar secas (tempos livres no meio do horário), criámos variáveis falsas para detetar isso. Se o professor tiver aula antes e depois do período X, mas no período X não tiver nada, isso conta como "1 buraco". Depois mandamos o solver tentar que essa conta dê o valor mais perto de zero possível. Nesta parte foi utilizado o auxilio de um LLM

Resultado: Buracos (Gaps) penalizados no horário: 0.0

4. Construção Incremental (Regra R9)

A nossa ideia: Se um professor ficar doente a meio do ano, não podemos deitar o horário todo da escola para o lixo e refazer do zero (isso seria um caos para os alunos). O que fizemos foi ler os dados novos (**dados_v2**) e usar o horário antigo como uma pista/palpite (*Warm-Start*). Dissemos ao solver: "tenta arranjar a confusão alterando o mínimo de aulas possível". Nesta parte foi utilizado o auxilio de um LLM

Comparação de Desempenho

Search...				
☐	Estratégia str	Tempo (s) float64	Aulas Alteradas object	
☐	Resolver do Zero (Mau)	0.3520	Máximas (Sem Controlo)	
☐	Usar a Solução Antiga (Bom)	0.2176	0	
2 rows, 3 columns No selection				

5. Conjunto de testes sob cada regra e Validação Automática

O que fazemos aqui: Não vamos confiar cegamente no computador. Criámos um "auditor" que pega no horário final e verifica, regra a regra, se há algum erro. Se o modelo falhar em alguma coisa, este teste vai apitá-lo. Também adicionamos uma turma fantasma para provar que o código aguenta crescer.

Resultado da Auditoria:  A auditoria testou as regras R1 a R7 com sucesso. Nenhum erro encontrado! O horário está perfeito.

Teste de Crescimento (Mais 1 Turma): O modelo adaptou-se automaticamente e a resolução deu Viável (Funcionou).

Horário da 7ªA

Search...						
☐	Index0 int64	Seg object	Ter object	Qua object	Qui object	
☐		1	Matemática (Sala Normal_1)	Matemática (Sala Normal_1)	Inglês (Sala Normal_2)	Ciências (Laborató
☐		2	-	-	-	-
☐		3	-	-	História (Sala Normal_6)	-
☐		4	Português (Sala Normal_3)	Português (Sala Normal_2)	Português (Sala Normal_3)	Matemática (Sala N
☐		5	Ciências (Laboratório_1)	Inglês (Sala Normal_1)	Matemática (Sala Normal_4)	Português (Sala No
5 rows, 5 columns No selection						

Horário da 7ªB

Search...

☐	Index0 int64	Seg object	Ter object	Qua object	Qui object
☐	1	-	-	-	-

☐	2	Ciências (Laboratório_1)	-	Inglês (Sala Normal_1)	-
☐	3	-	Ciências (Laboratório_1)	-	História (Sala Norm
☐	4	Matemática (Sala Normal_4)	Educação Física (Ginásio_1)	Matemática (Sala Normal_1)	Português (Sala No
☐	5	Português (Sala Normal_6)	Educação Física (Ginásio_1)	Português (Sala Normal_1)	Matemática (Sala N
5 rows, 5 columns No selection					

7. Documentação e Entrega

Porque escolhemos o OR-Tools (CP-SAT)?

Decidimos usar a biblioteca **CP-SAT Solver da Google OR-Tools** para resolver este trabalho prático de *Timetabling*. A Programação por Restrições (CP) foi a melhor opção porque nos deixou escrever regras de negócio diretas e lógicas (como obrigar aos blocos duplos contíguos na R4 e impedir furos temporais na R6) sem termos de usar matemática muito complicada. Além disso, ligou-se de forma muito fácil às tabelas de dados do Pandas que usámos.

Porque é que a Estratégia Incremental é importante (R9)?

No mundo real das escolas, há imprevistos. Se o programa refizesse o horário todo a partir do zero sempre que um professor ficasse de baixa de tarde, isso seria um pesadelo: os alunos viam as suas rotinas completamente baralhadas. Ao dizermos ao solver para olhar para o horário antigo como uma pista ("Warm-Start") e ao pedirmos para ele alterar o mínimo de aulas possível, conseguimos consertar o conflito como uma operação cirúrgica, afetando apenas as pessoas estritamente necessárias.

Uso de Ferramentas LLM (Registo de Diálogo)

O link para o diálogo integral com o LLM está disponível aqui: <https://share.gemini.google/bO0gYMhPdudN>

Durante o desenvolvimento, o uso do LLM focou-se em dois pilares avaliados:

- Adequação e Flexibilidade:** O LLM foi usado não para reescrever a lógica de negócio do zero, mas para integrar a lógica de CP-SAT (Warm-Start para H_1) dentro da arquitetura do Marimo. Ajudou-nos a adaptar a leitura dinâmica de duas fontes de dados diferentes (dados/ e dados_v2/), provando a flexibilidade do sistema a novos requisitos.
- Resolução de Problemas Específicos do Framework:** Devido à natureza reativa do Marimo, deparámo-nos com erros de redefinição de variáveis entre células (NameError). O LLM foi crucial para identificar o problema de âmbito (scope) das variáveis e reestruturar os ciclos de extração de dados sem comprometer a integridade do modelo matemático.
- **O modelo de linguagem (LLM) foi utilizado como ferramenta de suporte à depuração (debugging). A partir da explicação estrutural dos erros lógicos identificados na versão inicial do código, a solução final foi formulada e implementada de forma autónoma**